

edge-parking-lpr 제작 매뉴얼 — 빈 OpenSDK 앱에서 완성까지

대상 독자: Hanwha OpenSDK 앱을 처음 만드는 엔지니어. 이 문서만 따라가면 빈 샘플 앱에서 시작해 **온카메라 번호판 인식 → 주차 판정 → 전기차 실조회 → 위반 통지**까지 완결되는 옛지 앱을 재구축할 수 있다. 모든 수치·수칙은 실측으로 확정된 값이다.

항목	값
카메라	Hanwha PNM-C16083RVQ (멀티디렉셔널 4센서, Ambarella CV5)
플랫폼	OpenSDK 26.05.19 / SOC=cv5, 앱은 cgroup 컨테이너로 격리 (CPU 2코어)
동거 앱	WiseAI(감지·추적·크롭, NPU), DebugHelper, CloudConnector
OCR 모델	tinyLPR (noahzhy/KR_LPR) — 87KB TFLite, 입력 192×96 gray, CTC
최종 성능	오확정 0 / 실판글리프 판 생판독 conf 1.00 / entry→final 수 초

0. 설계 사상 — 시작 전에 읽을 것

- **카메라가 잘하는 건 카메라에게.** 감지·추적·베스트샷은 WiseAI(NPU)가 한다. 앱은 그 부산물 (메타데이터 XML)을 받아 **판독과 판정만** 한다. 2코어로 사는 유일한 방법.
- **앱은 영상을 디코드하지 않는다.** 30fps 텍스트(XML)를 소비하고, 필요한 순간에만 이미지 1장을 집어온다. (풀해상 JPEG 1장 디코드가 100~500ms — 영상 디코드는 불가능)
- **오독은 전제, 오확정은 금지.** 모델은 반드시 틀린다. 관문을 겹겹이 세우고 확신 없으면 침묵(HOLD)한다. 모든 문턱값은 실측 사고에서 나왔다.

1. 개발 환경 준비

1. 개발 PC: Windows + WSL2(Ubuntu) + Docker Desktop.
2. OpenSDK 배포판에서 도커 이미지 로드: `opensdk:26.05.19` (약 12GB).

3. 카메라와 같은 서브넷 (예: 카메라 192.168.0.5, PC 192.168.0.11). 계정은 digest 인증.

2. 빈 앱 골격 만들기

2.1 샘플 복제

SDK의 `SampleApplication/` 에서 메타데이터를 받는 샘플 하나를 통째로 복사해 프로젝트 폴더를 만든다. 핵심 디렉터리 구조:

```
project/
├── docker-compose.yml          # 빌드 진입점
├── app/
│   ├── CMakeLists.txt
│   ├── src/
│   │   ├── PLifeCycleManagemanifest.json  # ★ 매니페스트 "원본" (빌드가 app/bin 으로 복사)
│   │   ├── CMakeLists.txt
│   │   └── sample_component/
│   │       ├── sample_component.cc/.h      # 앱 본체 (SDK 접점)
│   │       ├── manifests/                  # 컴포넌트 매니페스트
│   │       ├── includes/                   # 도메인 모듈 (헤더 온리)
│   │       └── ocr_models/                  # 모델 · 라벨 · 등록명부 (res 로 패키징)
│   ├── html/index.html                 # 앱 웹 UI
│   ├── 3rd-party/                       # OpenCV 정적, tflite
│   └── libs/                             # 동봉 컴포넌트 (.so)
└── config/app_manifest.json             # AppName 선언
```

함정 #1 — 매니페스트 원본은 app/src 다. 빌드가 `app/src/PLifeCycleManagemanifest.json` → `app/bin/` 으로 재생성한다. `app/bin` 쪽만 고치면 다음 빌드가 조용히 되돌린다 (포트 설정으로 반나절 헤맨 실측).

2.2 앱 ID 규약

`config/app_manifest.json` 의 AppName 과 매니페스트의 ContainerName 이 앱 ID다. HTTP 경로 (`/opensdk/<AppID>/...`)와 EventStatus 라인에 그대로 박히므로 **한 번 정하면 바꾸지 않는다** (카메라 설치 식별자).

2.3 빌드와 설치

```
SDK_VER=26.05.19 SOC=cv5 APP_NAME=object_detect docker compose up # → object_detect.cap
```

설치: 카메라 웹 UI → 오픈플랫폼 → .cap 업로드 → Start.

함정 #2 — 설치 수칙. 반드시 **영상 정지 + 카메라 물리 재부팅** 직후 설치. 연속 설치하면 "반쯤-설치 오염"이 생긴다 — Status=Running 인데 HTTP 전부 500, 물리 재부팅으로만 회복.

3. 메타데이터 구독 — 앱의 눈

3.1 구독 선언

매니페스트의 `RemoteComponentNames` 에 채널별 메타데이터 매니저 스텝을 선언한다:

```
{ "LocalComponentName": "MetadataManager_{Ch}",  
  "ContainerName": "System",  
  "RemoteComponentName": "MetadataManager_{Ch}" }
```

이러면 SDK가 매 프레임 `eMetadataRequest` 이벤트로 ONVIF XML 을 배달해준다. `ProcessAEvent()` 에서 분기해 `HandleMetadataEvent → ProcessObjects(ch, xml)` 로 흘린다.

3.2 XML 파싱 (순수 함수로 격리)

파서는 헤더 하나(`core/metadata_parser.h`)에 가둔다 — 카메라를 바꾸면 이 파일만 교체. 추출하는 것:

- 객체 분류: Human / Vehicle / **LicensePlate** + 차종
- bbox (`<tt:Scale>` 로 좌표계 복원 → 0~1 정규화), CenterOfGravity
- 번호판 객체의 `Parent` (부모 차량 id)와 `<tt:ImageRef>` — 카메라가 이미 잘라 `/tmp/download/` 에 저장해둔 번호판 크롭 JPEG 경로. 이게 공짜 재료다.

3.3 WiseAI 채널 설정 (카메라 쪽)

- 객체 감지에서 **번호판 체크 필수** (고면 ImageRef 판 bbox 전부 끊김)
- 최소 객체 크기를 낮출 것 (기본 46px → 12px; 작은 모형/원거리 판 대응)
- 이 설정은 **채널별**이다 — 4채널 쓰려면 채널마다 복제

4. 판독 재료 — 크롭 2경로

4.1 good-shot (ImageRef, 정예 1장)

`core/plate_store.h`: ImageRef JPEG 을 읽어 링버퍼(`cap_<n>.jpg`, 100장)에 저장.

- 완성 검증: `FFD8 ... FFD9` 시그니처 — 카메라가 아직 쓰는 중인 부분파일이면 pending 등록 후 최대 3초 재시도
- 중복 제거: 같은 ImageRef 스킵, 같은 oid 는 같은 슬롯 덮어쓰기
- 경로에서 `objectid_<oid>` 파싱 — 판독 결과를 그 차의 투표함으로 배달하는 열쇠

4.2 버스트 (스냅샷 자체 크롭, 물량)

`RefreshSnapshot(ch)`: 카메라에 스냅샷 1장 주문(`eAppSnapshotJpeg`, 100ms 스로틀) → 완성 JPEG 을 메모리 캐시(`last_jpeg` + 세대번호 `jpeg_ver`). 매 프레임 번호판 bbox 로 풀해상(2592×1520) 스냅샷에서 직접 크롭 → 경량 OCR → 표 적립.

- 여백: bbox 가로 +35% / 세로 +60% (스냅샷 지연으로 bbox 가 어긋나는 것 흡수)
- 신선도: 같은 스냅샷 세대는 oid 당 1회만 판독 (낡은 프레임 재탕 = 유령 표)
- 디코드 캐시: 같은 세대는 1회만 imdecode (100ms급 비용)

합정 #3 — 센서 거울상. 이 카메라는 재부팅 상태에 따라 센서가 좌우반전으로 잡힐 수 있다. 판독 전량이 헛것이면 **원본 스냅샷에서 차 로고 방향부터 확인**하라. 로고가 뒤집혀 있으면 `kFlipCropH=true` (판독 직전 크롭 좌우반전). 거울상 크롭은 항상 완전 헛것을 뱉는다.

5. OCR 엔진 — tinyLPR 온보드

5.1 번들과 로드

- `ocr_models/`에 `kr_lpr.tflite` (87KB) + `kr_lpr_labels.txt` (문자 67개 = 숫자10 + 한글40 + 지역 토큰 17개: 서울·부산...제주가 각각 1클래스) 배치, CMake `install(DIRECTORY ...)` 로 res 패키징
- TFLite C API 로 로드. **스레드 1개 고정** — 2스레드는 스레드풀 스핀이 2코어를 독점해 앱 동결(실측)

5.2 판독 파이프라인 (`ocr/plate_ocr.h` Recognize)

1. (옵션) 좌우반전 보정 → 그레이
2. 판 색 분류 (밝은 픽셀 평균색 → 흰/노랑/연두/파랑) — 색-한글 규칙(영업용 한글 아바사자 배는 노랑판 전용)이 1차 환각 필터
3. **멀티후보 토너먼트**: 한 크롭을 4가지 재프레이밍(원본/타이트/중앙 2중)으로 전부 판독, "문법 유효 > 신뢰도"로 채택 — WiseAI 크롭 여백이 험령해 원본만 읽으면 17% 헛읽음(실측)
4. `conf < 0.95` 일 때만: 지터 앙상블($\pm 3\text{px}$)·조도 정규화(감마 LUT)·디블러 — **전부 결과 캡 0.94**

설계 헌법 — "선택 이후 `conf` 를 올리는 모든 가공은 0.94 캡". 가공본이 오답을 게이트 (0.95) 위로 부스트해 오확정시킨 사고가 4번 반복된 뒤 세운 규칙. 원본 판독만 단독으로 게이트를 넘을 자격이 있다.

5.3 디코드·문법

- 전처리: letterbox(비율 유지, 좌상단 정렬) $192 \times 96 \rightarrow /255$
- CTC: 스텝별 `argmax` → 연속중복 제거 → blank 제거. 신뢰도 = 비블랭크 확률의 기하평균
- 문법 검사: 숫자2~3 + 한글1 + 숫자4 (지역판은 지역토큰 접두). 무효 텍스트는 어느 관문에도 못 들어감

모형(토이카) 리그 제1 조건 — 폰트. `tinyLPR` 는 실판 전용 서체로만 학습됐다. 둥근 범용 폰트 인쇄물은 `sharp 2000+` 로 선명해도 체계 오독(9→0)한다. 반대로 실판 글리프는 강블러·저해상·원근에도 정독 (같은 판에서 `conf` 0.80→1.00 실측). 인쇄 시트는 학습 데이터 생성기 (`kr_plate_gen`)의 글리프로 만들 것. 크기 하한: 판이 프레임에서 ~150px 폭 이상.

6. 판정 — 오확정 0의 구조

6.1 투표함 (ocr/plate_vote.h)

차(oid)별로 good-shot(primary)-버스트 표를 모은다. 개표 점수 = $\max(\text{conf}) + 0.02 \times \min(\text{표수}-1, 3) + \text{primary } 0.02$. 한글만 다른 접전은 캡 0.94 (tinyLPR 최대 약점이 한글 음절), 교차합의(굿샷+버스트 동일 2표+)는 0.97 승격.

6.2 3단 판정

단계	조건	결과
1단 즉시확정	good-shot 원본 conf ≥ 0.985 & 문법 유효 & 무반박(0.98+) ※ 구역 있는 채널은 단발 확정 금지 — 표로 넣고 즉석 개표(합의제)	★FINAL
2단 합의	2표 합의 + 게이트 conf ≥ 0.95 (등록차 정확일치 ≥ 0.97 은 단표 db-instant)	★FINAL
3단 DB 그물	등록명부와 편집거리 ≤ 1 유일(0.80/0.85) / 같은 길이 치환 ≤ 2 유일(0.90)	★FINAL (db-rescue/-fix)
전부 미달	—	?HOLD (침묵)

6.3 등록명부 (registered_plates.txt)

한 줄 1판. 실전의 "아파트 등록차 목록"에 해당하는 정당한 레이어 — 역할은 번호 교정뿐, EV 여부의 출처가 아니다. 코드에 정답 하드코딩 금지.

6.4 지역판(택시) — 명부 지역복원

1단(왼쪽 세로 지역명) 레이아웃은 모델 미학습 — 꼬리만 읽히거나 엉뚱한 지역토큰이 붙는다. 이미지로 지역을 읽으려는 시도(NCC-재조립)는 실측 불안정으로 폐기. 대신:

- RecoverRegion: 선두 지역토큰 스트립 → 꼬리를 명부의 지역판과 치환 ≤ 2 유일 대조 → 완성 번호로 승격 (db-region; 게이트 미달은 오차별 0.85/0.90 문턱으로 rescue)
- 미검증 지역 = 확정 금지: 복원 안 되는 지역 접두 판독은 FINAL 자격 박탈(hold)

7. 주차 판정 (core/parking_zone.h)

7.1 구역

4점 폴리곤. 웹 UI 클릭 4번 또는 `POST /parking_roi?ch=N` (본문 = 8숫자, 0~1 정규화). 파일로 영속화 — 재부팅·재설치에도 유지.

7.2 상태기계

빈칸 → 후보(차량 겹침65% or 구역 안 번호판) -2초 정지 → PARKED → LEFT → 빈칸(+유령 15초)

- 점유 유지 증거: 칸을 덮는 "정지" 차량(id 무관 — WiseAI id 뽕뽕이 흡수) 또는 번호판 중심점
- 판 증거 배타 귀속: 판 중심점은 "확장 bbox(+burst_margin) 안 + 최근접 칸 하나"에만 귀속 — InPoly 강짜는 구역을 판보다 작게 그린 현실에서 증거를 전멸시킴(실측), 무제한 확장은 옆 칸 도둑질
- 출차 2경로: 겹침≤10% 이탈 = 즉시 / 부재 지속 = 유예(grace) 후 육안검증(스냅샷으로 그 칸에 같은 판이 남았는지 확인, 유사판정 ≤2글자) N연속 빈칸 → LEFT
- 완결 칸(점유+번호+EV)은 출차까지 판독 완전 침묵 (NeedsRead 게이트)
- 유령 명부: 출차한 차의 번호판 id 를 15초간 점유 증거·ImageRef 저장/판독에서 제외 — WiseAI 가 차 교체 후 트랙 id 를 유지하면 best-shot 이 이전 차의 판이라 오배정된다(실측)

7.3 위반

위반 = 점유 $\wedge$ 비EV. EV 미확정(unknown)은 위반으로 치지 않는다 (억울한 알람 방지).

8. 전기차 실조회 (io/ev_client.h)

함정 #4 — 앱 내 TLS 금지. 앱 프로세스에서 OpenSSL 로 TLS 를 열면 첫 SSL 호출에서 **앱이** 즉사한다 (플랫폼 자체 TLS 와 동거 불가, 실측 2회). 외부 HTTPS 는 `popen` 으로 카메라 내장 `curl/wget` 자식 프로세스를 돌려라. CMake 에 OpenSSL 링크 금지.

- 조회: ev.or.kr 무공해차 확인 `POST selectCarNum=1&searchWord=<번호>` → 1종 = EV. 엔드포인트는 `config.h` 단일 출처 (하드코딩 금지)
- 전용 워커 스레드 — 메타데이터 파이프라인을 절대 막지 않는다. 결과는 큐 → 메인 스레드가 회수

- 등록차도 실조회가 진실 — 명부 플래그는 조회 실패 시 폴백만 (플래그는 낡는다, 실측)

9. HTTP API 만들기

2단계다: ① 초기화 때 경로 등록, ② 요청 이벤트 처리.

```
// ① 등록 (쓰기 있으면 메서드에 POST 포함 - 빼먹으면 405 인데 앱 로그 무늬적)
auto* r = new ("OpenAPI") IAppDispatcher::OpenAPIRegistrar(
    String("/parking_tune"), GetInstanceName(), methods);
SendNoReplyEvent("AppDispatcher", eRegisterCommand, 0, r);

// ② 처리
std::string path = oas->GetFCGXParam("PATH_INFO").c_str();
if (path == "/parking_tune") { ... oas->SetResponseBody(json, len); }
```

- 경로는 한 세그먼트, 하위는 쿼리스트링(?ch=0). JSON 파싱은 수동(find+atof)
- `SetStatusCode` 는 본문 설정 **전에**
- 핸들러는 짧게 — 메타데이터와 같은 스레드 계열. 무거운 일은 상태만 바꾸고 유지보수 루프에 넘김
- POST 응답에 최종값 전체를 돌려줘 클라이언트가 반영 여부를 그 자리에서 확인

주요 엔드포인트

경로	역할
GET /parking_status	칸별 상태 XML (번호·EV·위반·증거URL·좌표) — 외부 연동 서류철
GET/POST /parking_roi	구역 등록/삭제
GET/POST /parking_tune	판정 파라미터 12종 런타임 튜닝 (부분갱신·범위검증·영속화) — 튜닝 사이클 10분→3초
GET /snapshot?ch=	라이브 프레임 (UI 스트리밍)
GET /final?n= /finallist	FINAL 에 실제 사용된 이미지 전용 갤러리 (증거 = 판정 근거 사진)
GET /isev?plate=	등록·EV·목격 여부 JSON
GET /eventlog	전체 이벤트 이력 (디버깅)

10. EventStatus 커스텀 이벤트 (벨)

외부 통지는 **notify-then-fetch** 2채널: 벨(불리언 푸시) + 서류철(XML 풀). 이벤트 채널은 펌웨어 제약으로 불리언만 실린다 — 그래서 이 구조가 강제된다.

1. **중계자 번들**: SDK 의 OpenEventDispatcher(.so+manifest)를 `app/libs/` 에 동봉
2. **스텝 선언**: 매니페스트에 `Stub::Dispatcher::Eventstatus → EventstatusCGIDispatcher`
3. **스키마 응답**: 부팅 시 펌웨어의 3종 질의에 응답 — 이게 상황판에 우리 블록을 추가하는 행위
 - `eEventstatusSchema` : `Channel.<int>.OpenSDK.<앱>.ParkingOccupied[.<int>]`
 - `eMetadataSchema` : ONVIF 토픽 선언
 - `eEventStatusCheck` : 현재 상태 스냅샷
4. **변화 통지**: 상태 변화 시 `eEventStatusChanged` 를 OpenEventDispatcher 로 발송. 값이 안 변하는 내용 갱신(번호 확정 등)은 **False→True 펄스로** diff 라인을 강제

클라이언트 규칙은 3줄: *"Occupied 라인이 오면 (값 무관) XML 을 읽고 violation 필드로 판단하라."*

11. 웹 UI

- `app/html/index.html` 한 장 — cap 에 패키징되어 `/opensdk/<앱>/html/` 로 서빙
- base URI 는 경로에서 추출: `location.pathname.split('/')[5] = AppID`
- 구성: 채널 탭 / 스냅샷 스트리밍(더블 버퍼) / 감지박스.구역 캔버스 오버레이 / 구역 그리기 (4클릭) / 튜닝 패널 / FINAL 증거 갤러리
- 탭이 안 보이면(document.hidden) 모든 폴링 중단 — 카메라 부하 0

12. 디버깅 체계

도구	비고
remote_debug_viewer (SDK CLI)	앱 매니페스트(app/src)에 <code>SkeletonPortNumber: 8590</code> 고정 + 뷰어 설정의 해당 앱 PortNumber 를 8590 으로 일치. auto 면 뷰어가 포트를 몰라 <code>port # 0</code> 실패

GET /eventlog	같은 이벤트를 HTTP 로 — 포트 설정 불필요, 항상 동작
이벤트 sink 패턴	도메인 코드는 <code>EmitEvent()</code> 한 줄만 — <code>DebugViewerSink</code> (원격) + <code>DiskLogSink(events.log)</code> 로 팬아웃. 출구 추가 = sink 한 줄
계측 로그	<code>cycle</code> (entry→parked→first-read→final), <code>decode</code> (횟수·평균ms), <code>burst reject</code> (게이트 미달 판독 가시화 — "왜 안 읽히나"를 장님으로 만들지 않기)

13. 실측 함정 총정리 (재시도 금지 목록)

#	함정	결론
1	매니페스트를 app/bin 에서 수정	빌드가 app/src 원본으로 덮음 — 항상 src 수정
2	연속 설치	반쯤-설치 오염 (Running 인데 HTTP 500) — 영상정지+재부팅 후 설치
3	앱 내 OpenSSL TLS	즉사 — <code>popen curl</code> 자식 프로세스
4	TFLite 2스레드	스핀 동결 — 1스레드 고정, 속도는 "일 줄이기"로만
5	둥근 PC폰트 인쇄판	선명해도 체계 오독 — 실판 글리프만
6	거울상 미확인	전량 헛것 — 스냅샷의 로고 방향으로 판별
7	가공 판독 무캡 승격	오답이 게이트 뚫음 (4회 반복) — 캡 0.94
8	업스케일/SR/디노이즈로 오독 복구	정보가 안 늘어 전패 (0/30, 0/14, 0/56 실측)
9	크롭 품질 필터 문턱을 감으로	실크롭까지 죽여 전량 굶김 — 실측 분포에서 문턱 도출
10	판독한 크롭 파일 재저장(가공)	재판독마다 재가공되는 자기파괴 루프 — 저장본은 불변으로
11	출차 후 ImageRef 신뢰	WiseAI 트랙 잔존 시 이전 차 best-shot — 유령 명부로 차단
12	미검증 지역토큰 확정	미학습 레이아웃의 지역토큰은 환각 — 명부 복원분만 확정

14. 부록 — 체크리스트

새 카메라 셋업

1. 카메라 화질·AI 골든 파라미터 적용 (CAMERA_SETTINGS.md)
2. WiseAI: 번호판 체크 + 최소 객체 12px (채널별)
3. cap 설치 (영상정지 → 업로드 → 재부팅 → Start)
4. 웹 UI 에서 채널별 주차구역 그리기 (판 위치가 폴리곤에 들어오게)
5. 등록명부 갱신 → 재빌드·재설치
6. 스냅샷에서 로고 방향 확인 → kFlipCropH 결정

런타임 튜닝 기본값

키	기본	의미
overlap	0.65	진입/주차 후보 겹침 문턱
dwel_ms	2000	정지 유지 → 주차 확정
grace_ms	1000~2000	부재 유예 (실제 출차는 이탈로 즉시)
exit_cover	0.10	겹침 이하 = 즉시 출차
presence_miss / period / conf	2 / 800 / 0.60	출차 육안검증 (실판글리프 전제)
burst_margin	0.35	판독·배정·귀속 공통 반경
best_frame_mode	0	1 = 챔피언 1장 모드 (채널당 1대 전제)
best_min_sharp / aspect	150 / 2.0	크롭 품질 필터 (블러·오크롭 컷)

관련 문서: LOGIC_TOUR(다이어그램 투어) · ARCHITECTURE(모듈 해부) · APP_HTTP_API(API 저작 상세) · PARKING_EVENTS(외부 연동 명세) · CAMERA_SETTINGS(골든 파라미터)